Introduction to Parallel Algorithms and Programming

Session 7: Distributed-memory programming with MPI

Oguz Kaya

Assistant Professor
Université Paris-Saclay and LISN, Gif-sur-Yvette, France

université PARIS-SACLAY

What we learned in session 6: algorithms on a ring

- **Distributed memory**: p processors, each with its **own memory**, linked by a network; no shared variable, the data moves by **messages**
- The **unidirectional ring**: P_k sends to P_{k+1} and receives from P_{k-1}, with four routines: MyRank(), NumProcs(), Send(msg, m), Recv(msg, m); Send does not block, Recv waits for the message
- **One program, many processes**: every processor runs the same program, **asynchronously**; its rank decides what it does (**if** $q = k \ldots$) and which elements it holds: a block of a global array
- A message of m elements costs $L + m\,b$; algorithms: Bcast, Reduce, 1-D convolution, Normalize, and those of TD2
- Today: the same programs **for real**, in C++ with MPI

Bcast(k, msg, m)

```
1: q ← MyRank(); p ← NumProcs()
2: if q = k then
3:     Send(msg, m)
4: else if q = (k − 1) mod p then
5:     Recv(msg, m)
6: else
7:     Recv(msg, m)
8:     Send(msg, m)
```

Objectives

- What a **process** is, how it differs from a thread (session 3), and why processes let us compute on **many machines** at once
- What **MPI** is: functions for exchanging messages between processes, and a runtime that launches them
- The minimal MPI toolbox, **six functions**:
  - start and stop: `MPI_Init`, `MPI_Finalize`
  - who am I, how many are we: `MPI_Comm_rank`, `MPI_Comm_size`
  - exchanging data: `MPI_Send`, `MPI_Recv`
- Two complete programs, step by step: computing nine with three processes, normalizing a distributed vector
- The dictionary PRAM / OpenMP / CUDA / MPI: the same ideas, a fourth syntax; how to compile and run
- Not today: collective operations (`MPI_Bcast`, `MPI_Reduce`, ...), non-blocking messages, communicators, ...: the MPI course of T3

Outline

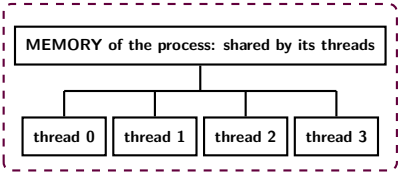
1. Processes, and the MPI library
2. Computing nine with three processes
3. Second example: normalizing a distributed vector
4. Summary
5. Compiling and running; the lab

Outline

1. Processes, and the MPI library
2. Computing nine with three processes
3. Second example: normalizing a distributed vector
4. Summary
5. Compiling and running; the lab

Processes and threads

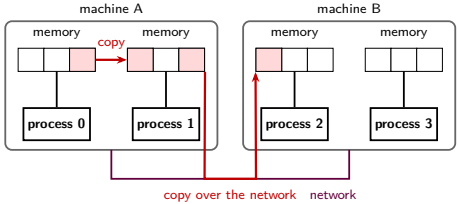
- `./program` launches the program as a **process**: its own memory (code, variables, heap) and one execution stream, which starts at `main`
- Inside the process, we can create several **threads**, with OpenMP for instance (session 3, `#pragma omp parallel`): they all read and write **the same memory**, that of their process
- The operating system runs the threads on the cores of the machine: P threads on P cores work simultaneously, on shared data



one process, `./program`, four threads created inside it (`#pragma omp parallel`)

Processes have private memories

- A process has a **private address space**: two processes **cannot touch each other's memory**, not even by mistake (segmentation fault); the operating system guarantees it
- So two processes need to share nothing: they can run on the same machine, one per core, or on **different machines**: a cluster, a supercomputer
- The price: **no shared variable at all**. To coordinate and to exchange data, processes send each other **messages** through the network: the distributed memory of session 6
- Two processes on the **same machine**: the data must still be **copied** from the memory of one into the memory of the other. A message is a copy, over the network or inside the machine



four processes (`mpirun -np 4`): private memories, on one machine or on several; a message is a copy of a block

What is MPI?

- **Message Passing Interface**: a standard library (1994) for exchanging messages between processes, with a **simple C interface**: functions, no new keyword, no pragma. Open MPI, MPICH, ...; C, C++, Fortran, Python
- Two things:
  - **functions**, declared in `<mpi.h>`: send a message, receive one, who am I, how many are we
  - a **runtime**: `mpirun -np 3 ./prog` launches 3 processes running `prog`, on one machine or on several, and connects them so that they can talk
- **One program, many processes** (session 6): every process runs the same program with its own **rank**, $0, \ldots, p-1$, which decides what it does. PU i executing the same program on index i, once more: MPI process $\leftrightarrow$ OpenMP thread $\leftrightarrow$ CUDA thread $\leftrightarrow$ PU
- Every supercomputer is programmed this way, from 2 to a million processes. MPI is large (400+ functions); today **six**, enough for every algorithm of session 6 and TD2



a supercomputer: tens of thousands of machines, one MPI program

Outline

1. Processes, and the MPI library
2. Computing nine with three processes
3. Second example: normalizing a distributed vector
4. Summary
5. Compiling and running; the lab

Computing nine, launched three times (01-nine-seq.cpp)


```
11 #define UNIT 100000 // 1 unit = 0.1 s (in microseconds)
12 int computeTwo()   { usleep(2 * UNIT); return 2; }
13 int computeThree() { usleep(3 * UNIT); return 3; }
14 int computeFour()  { usleep(4 * UNIT); return 4; }
15
16 int main()
17 {
18   double t0 = now();
19
20   int two = computeTwo();
21   int three = computeThree();
22   int four = computeFour();
23   int nine = two + three + four;
24
25   double t1 = now();
26   printf("nine = %d, computed in %.2f s\n",
27          nine, t1 - t0);
28   return 0;
29 }
```

- Session 3's example: three **independent** tasks of 2, 3 and 4 units of time (0.1 s of sleep each), returning 2, 3, 4; sequentially 9 units
- This program has **no MPI in it**. Launch it three times at once: `mpirun -np 3 ./01-nine-seq` creates **three processes**, each running the whole program
- Output: three lines `nine = 9, computed in 0.90 s`. Correct, and **no faster**: each process did exactly the same thing, in its own memory (session 3's redundant attempt)
- To go faster, each process must do a **different part** of the work: it needs to know **who it is**

Distributing the work by rank (02-nine-rank.cpp)

université PARIS-SACLAY

```
 4 #include <mpi.h>

           ...

17 int main(int argc, char **argv)
18 {
19   MPI_Init(&argc, &argv); // the first MPI call
20   int rank, size;
21   MPI_Comm_rank(MPI_COMM_WORLD, &rank); // my rank
22   MPI_Comm_size(MPI_COMM_WORLD, &size); // how many
23   double t0 = now();
24
25   int two = 0, three = 0, four = 0;
26   if (rank == 0) two = computeTwo();
27   if (rank == 1) three = computeThree();
28   if (rank == 2) four = computeFour();
29   int nine = two + three + four;
30
31   double t1 = now();
32   printf("process %d: nine = %d, computed in %.2f s\n",
33          rank, nine, t1 - t0);
34   MPI_Finalize(); // the last MPI call
35   return 0;
36 }
```

- Line 4: `mpi.h` declares the MPI functions
- Lines 19 and 34: `MPI_Init` and `MPI_Finalize` start and stop the communication among the processes `mpirun` created; **every MPI call lies between the two**
- Lines 21–22: my **rank**, $0 \ldots \texttt{size} - 1$, and the number of processes: MyRank(), NumProcs() of session 6; `MPI_COMM_WORLD` = all the processes
- Lines 26–28: work **dispatched by rank**, as `if (thid == 0)` in session 3. Output: `nine` = 2 (0.20 s), 3 (0.30 s), 4 (0.40 s): **wrong**. Each process holds **one third of the data**: every variable exists in three copies, and nobody sees the others'

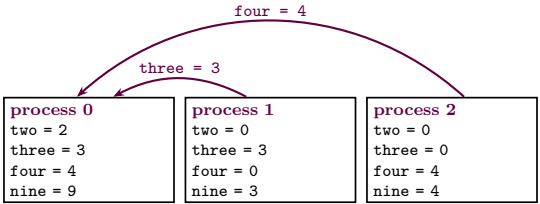
Exchanging data: MPI_Send and MPI_Recv


```
MPI_Send(buf, count, type, dest,   0, MPI_COMM_WORLD);
MPI_Recv(buf, count, type, source, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
```

- A **message** is an array `buf` of `count` elements of a basic `type` (`MPI_INT`, `MPI_FLOAT`, `MPI_DOUBLE`, ...), sent by one process to another: session 6's SEND(msg, m) and RECV(msg, m)
- `dest`, the rank of the receiver, is specified by the sender; `source`, the rank of the sender, by the receiver. Unlike the unidirectional ring of session 6, **any process can send directly to any other**: MPI moves the data underneath, routed through the network (the SENDTO of TD2) or, on the same machine, simply copied from the address space of one process into the other's; we do not deal with that
- The last two arguments of `MPI_Send` and the last three of `MPI_Recv` are always these: ignore them, they are outside this crash course
- A **single variable**, not an array? It is an array of one element: its address, and `count = 1`: `MPI_Send(&three, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);`
- A message is written **on both sides**, as a **matching** send and receive: the same `count` and `type`, `dest` naming the receiver and `source` the sender, and **both calls made**: if a process never reaches its `MPI_Send`, the `MPI_Recv` on the other side waits forever

Merging the results in process 0 (03-nine-merge.cpp)

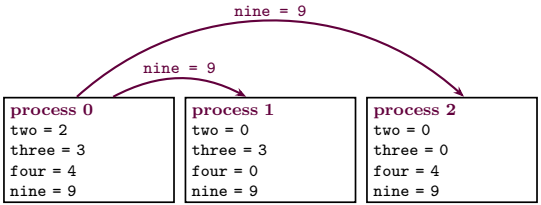

```
31  if (rank == 1)
32    MPI_Send(&three, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);
33  if (rank == 2)
34    MPI_Send(&four, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);
35  if (rank == 0) {
36    MPI_Recv(&three, 1, MPI_INT, 1, 0, MPI_COMM_WORLD,
37             MPI_STATUS_IGNORE);
38    MPI_Recv(&four, 1, MPI_INT, 2, 0, MPI_COMM_WORLD,
39             MPI_STATUS_IGNORE);
40  }
41  int nine = two + three + four;
```



- Lines 31–34: processes 1 and 2 **send** their value to process 0 (one `int`)
- Lines 35–40: process 0 **receives** process 1's message into its own `three`, process 2's into its own `four`: its three variables are set
- Line 41: `nine = 9` on process 0, still 3 and 4 elsewhere. Output: `process 0: nine = 9, computed in 0.40 s`: the longest task, speedup 2.25 (session 3)
- `-np 2`: process 2 does not exist, the receive from rank 2 is an **error**, MPI aborts (session 3 printed 5 silently); `-np 4`: process 3 idles and prints 0

Everybody gets the result (04-nine-bcast.cpp)


```
44  if (rank == 0) {
45    MPI_Send(&nine, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
46    MPI_Send(&nine, 1, MPI_INT, 2, 0, MPI_COMM_WORLD);
47  }
48  if (rank == 1 || rank == 2)
49    MPI_Recv(&nine, 1, MPI_INT, 0, 0, MPI_COMM_WORLD,
50             MPI_STATUS_IGNORE);
51
52  double t1 = now();
53  printf("process %d: nine = %d, computed in %.2f s\n",
54         rank, nine, t1 - t0);
```



- The result lives in process 0 only. Lines 45–46: process 0 **sends** `nine` to process 1, then to process 2; lines 48–50: they **receive** it into their own `nine`
- The **mirror** of the previous slide: the same lines with `MPI_Send` and `MPI_Recv` swapped; every send has its receive
- Output: three lines `nine = 9, computed in 0.40 s`: everybody is right, at the time of the longest task
- Session 6's **Reduce** then **Bcast**, by hand, as a **star** around process 0 rather than along a ring: any process can talk to any other

The whole program (04-nine-bcast.cpp)


```
#include <cstdio>
#include <unistd.h>
#include <chrono>
#include <mpi.h>

          ... lines 5-16:  now() and the three tasks

int main(int argc, char **argv)
{
  MPI_Init(&argc, &argv);
  int rank, size;
  MPI_Comm_rank(MPI_COMM_WORLD, &rank);
  MPI_Comm_size(MPI_COMM_WORLD, &size);
  double t0 = now();

  int two = 0, three = 0, four = 0;
  if (rank == 0) two = computeTwo();
  if (rank == 1) three = computeThree();
  if (rank == 2) four = computeFour();

  // the partial results are merged in process 0
  if (rank == 1)
    MPI_Send(&three, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);
  if (rank == 2)
    MPI_Send(&four, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);
  if (rank == 0) {
    MPI_Recv(&three, 1, MPI_INT, 1, 0, MPI_COMM_WORLD,
             MPI_STATUS_IGNORE);
    MPI_Recv(&four, 1, MPI_INT, 2, 0, MPI_COMM_WORLD,
             MPI_STATUS_IGNORE);
  }
  int nine = two + three + four;

  // the mirror: process 0 sends the result to the others
  if (rank == 0) {
    MPI_Send(&nine, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
    MPI_Send(&nine, 1, MPI_INT, 2, 0, MPI_COMM_WORLD);
  }
  if (rank == 1 || rank == 2)
    MPI_Recv(&nine, 1, MPI_INT, 0, 0, MPI_COMM_WORLD,
             MPI_STATUS_IGNORE);

  double t1 = now();
  printf("process %d: nine = %d, computed in %.2f s\n",
         rank, nine, t1 - t0);
  MPI_Finalize();
  return 0;
}
```

Six functions are enough

Function	What it does
`MPI_Init(&argc, &argv)`	Starts MPI: connects the processes `mpirun` launched; the first MPI call
`MPI_Finalize()`	Stops MPI; the last MPI call
`MPI_Comm_rank(MPI_COMM_WORLD, &rank)`	My rank, an `int` between 0 and $p-1$: session 6's MyRank()
`MPI_Comm_size(MPI_COMM_WORLD, &size)`	The number p of processes: NumProcs()
`MPI_Send(buf, count, type, dest, 0, MPI_COMM_WORLD)`	Sends the `count` elements of type `type` (`MPI_INT`, `MPI_FLOAT`, ...) of `buf` to process `dest`: Send(msg, m) to a chosen process
`MPI_Recv(buf, count, type, source, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE)`	Receives into `buf` (room for `count` elements) the message of process `source`: Recv(msg, m) from a chosen process

- **This is all we need**: today, in the lab, for every algorithm of session 6 and TD2. Every function returns an error code, ignored; results come back through pointers (`&rank`)
- As in session 6, `MPI_Send` **does not block**: it returns at once and the message makes its way — unless the message is big, a case we skip (it needs `MPI_Isend`: the MPI course)
- `MPI_Recv` **blocks** until the message has arrived: the only wait; otherwise the processes are **asynchronous**

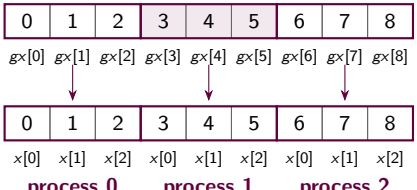
Outline

1. Processes, and the MPI library
2. Computing nine with three processes
3. Second example: normalizing a distributed vector
4. Summary
5. Compiling and running; the lab

A distributed vector

Every process begins the program with a `float` vector `x` of n elements. Think of the P blocks as one **global vector** $gx[0 \dots N-1]$, $N = nP$ (session 6): $gx[k\,n + j]$ is the $x[j]$ of process k. In the example, $gx[i] = i$.

the global vector $gx[0 \dots N-1]$, $N = nP = 9$: nobody holds it, it exists only in our head



$gx[k\,n + j]$ is the $x[j]$ of process k: the $x[1]$ of process 2 is $gx[7]$

- Process k holds the elements $k\,n, \dots, (k+1)\,n - 1$ and knows **nothing** of the others
- We want to **normalize** gx: $x[j] \leftarrow x[j] \,/\, \|gx\|_2$ on every process, with $\|gx\|_2 = \sqrt{gx[0]^2 + \dots + gx[N-1]^2}$; here $\sqrt{0 + 1 + 4 + \dots + 64} = \sqrt{204} \approx 14.28$
- Every new value depends on **all** the N elements, and each process has n of them. **How?**

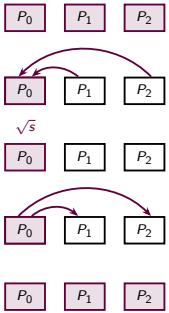
Normalizing a vector: the algorithm of session 6

université PARIS-SACLAY

NORMALIZE(x, n)

```
1: q ← MyRank()
2: s ← 0
3: for j = 0, ..., n − 1 do
4:    s ← s + x[j] · x[j]   ▶ local sum of squares
5: Reduce(0, s, 1, +)        ▶ P0: s = Σ_{i=0}^{N−1} gx[i]^2
6: if q = 0 then s ← √s      ▶ the norm
7: Bcast(0, s, 1)            ▶ all: s = ‖gx‖2
8: for j = 0, ..., n − 1 do
9:    x[j] ← x[j]/s
```

Session 6's REDUCE and BCAST went around the ring; here we write them by hand, with messages straight to and from process 0: a **star**, since any process can talk to any other.



1. local sums of squares (lines 3–4)
2. the sums merged in P_0 (REDUCE, line 5)
3. P_0 computes the norm (line 6)
4. the norm sent to all (BCAST, line 7)
5. local divisions by the norm (lines 8–9)

Step 1: the sum of the squares of my block


```
19  MPI_Init(&argc, &argv);
20  int rank, size;
21  MPI_Comm_rank(MPI_COMM_WORLD, &rank);
22  MPI_Comm_size(MPI_COMM_WORLD, &size);
23  float *x; int n; // my block of the global vector
24  init(&x, &n);    // its n elements, see init above
25
26  // step 1: the sum of the squares of my block
27  float s = 0;
28  for (int j = 0; j < n; j++) s = s + x[j] * x[j];
```

process 0	process 1	process 2
x = 0 1 2	x = 3 4 5	x = 6 7 8
s = 5	s = 50	s = 149

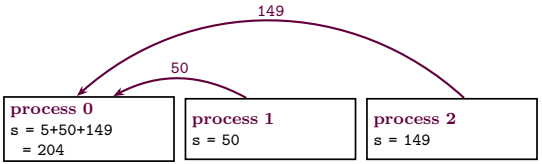
NORMALIZE(x, n) of session 6, this step

1: $q \leftarrow$ MYRANK()
2: $s \leftarrow 0$
3: **for** $j = 0, \ldots, n-1$ **do**
4: $\quad s \leftarrow s + x[j] \cdot x[j]$

- Lines 19–22: the start of every MPI program; line 21 is line 1 of the algorithm, $q \leftarrow$ MYRANK()
- Lines 23–24: my block x of n elements, from `init`: here $n = 3$ and $gx[i] = i$, so process 0 holds 0, 1, 2, process 1 holds 3, 4, 5, process 2 holds 6, 7, 8
- Lines 27–28: lines 2–4 of the algorithm, local, **no communication**

Step 2: merging the sums in process 0


```
30  // step 2: process 0 adds up the sums of the others
31  if (rank == 0) {
32    for (int k = 1; k < size; k++) {
33      float t;
34      MPI_Recv(&t, 1, MPI_FLOAT, k, 0, MPI_COMM_WORLD,
35               MPI_STATUS_IGNORE);
36      s = s + t;
37    }
38  } else {
39    MPI_Send(&s, 1, MPI_FLOAT, 0, 0, MPI_COMM_WORLD);
40  }
```



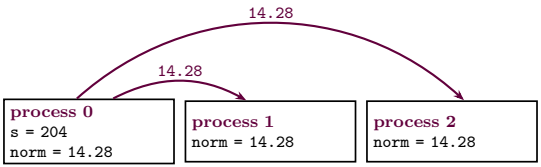
NORMALIZE(x, n) of session 6, this step

5: REDUCE$(0, s, 1, +)$ ▶ P_0: the total

- Line 5 of the algorithm, by hand. Line 39: every process but 0 **sends** its `s` to process 0
- Lines 32–37: process 0 **receives** $P - 1$ messages, from ranks $1, \ldots, P - 1$ in turn, into `t`, and accumulates: $5 + 50 + 149 = 204$
- The `if`/`else` on the rank holds the **two sides of the conversation**. Process 2's message may arrive before process 1's: it waits in MPI's hands until line 34

Step 3: the norm goes back to everybody


```
42  // step 3: process 0 computes the norm, sends it out
43  float norm;
44  if (rank == 0) {
45    norm = sqrt(s);
46    for (int k = 1; k < size; k++)
47      MPI_Send(&norm, 1, MPI_FLOAT, k, 0, MPI_COMM_WORLD);
48  } else {
49    MPI_Recv(&norm, 1, MPI_FLOAT, 0, 0, MPI_COMM_WORLD,
50             MPI_STATUS_IGNORE);
51  }
```



NORMALIZE(x, n) of session 6, this step

6: **if** $q = 0$ **then** $s \leftarrow \sqrt{s}$ ▶ the norm
7: BCAST$(0, s, 1)$ ▶ all: $s = \|x\|_2$

- Line 6: lines 44–45, process 0 **alone** takes the square root, $\sqrt{204} \approx 14.28$, the norm of the whole vector
- Line 7 by hand: lines 46–47, process 0 **sends** it to each rank $k = 1, \ldots, P-1$; lines 49–50, the others **receive** it into their own `norm`
- The **mirror** of step 2: the loop of sends answers the loop of receives, the single receive answers the single send

Step 4: every process normalizes its own block


```
53   // step 4: every process normalizes its own block
54   for (int j = 0; j < n; j++) x[j] = x[j] / norm;
55   printf("process %d: norm = %.2f, x =", rank, norm);
56   for (int j = 0; j < n; j++) printf(" %.2f", x[j]);
57   printf("\n");
58
59   free(x);
60   MPI_Finalize();
61   return 0;
62 }
```

process 0	process 1	process 2
norm = 14.28	norm = 14.28	norm = 14.28
x = 0.00 0.07 0.14	x = 0.21 0.28 0.35	x = 0.42 0.49 0.56

NORMALIZE(x, n) of session 6, this step

8: **for** $j = 0, \ldots, n-1$ **do**
9: $\quad x[j] \leftarrow x[j]/s$

- Lines 8–9: line 54, n divisions on the local block, **no communication**
- Output: `process 0: norm = 14.28, x = 0.00 0.07 0.14`, then `0.21 0.28 0.35` and `0.42 0.49 0.56` for processes 1 and 2, in any order
- The global vector has been normalized although **nobody ever held it**: $2(P-1)$ messages of one element

The whole program (05-normalize.cpp)


```
1  #include <cstdio>
2  #include <cstdlib>
3  #include <cmath>
4  #include <mpi.h>
5
6  // my block: n elements of a global vector gx, gx[i] = i,
7  // N = n * size elements in all, gx[k*n] first on process k
8  void init(float **x, int *n)
9  {
10   int rank;
11   MPI_Comm_rank(MPI_COMM_WORLD, &rank);
12   *n = 3;
13   *x = (float *)malloc(*n * sizeof(float));
14   for (int j = 0; j < *n; j++) (*x)[j] = rank * (*n) + j;
15 }
16
17 int main(int argc, char **argv)
18 {
19   MPI_Init(&argc, &argv);
20   int rank, size;
21   MPI_Comm_rank(MPI_COMM_WORLD, &rank);
22   MPI_Comm_size(MPI_COMM_WORLD, &size);
23   float *x; int n; // my block of the global vector
24   init(&x, &n);    // its n elements, see init above
25
26   // step 1: the sum of the squares of my block
27   float s = 0;
28   for (int j = 0; j < n; j++) s = s + x[j] * x[j];
29
30   // step 2: process 0 adds up the sums of the others
31   if (rank == 0) {
32     for (int k = 1; k < size; k++) {
33       float t;
34       MPI_Recv(&t, 1, MPI_FLOAT, k, 0, MPI_COMM_WORLD,
35                MPI_STATUS_IGNORE);
36       s = s + t;
37     }
38   } else {
39     MPI_Send(&s, 1, MPI_FLOAT, 0, 0, MPI_COMM_WORLD);
40   }
41
42   // step 3: process 0 computes the norm, sends it out
43   float norm;
44   if (rank == 0) {
45     norm = sqrt(s);
46     for (int k = 1; k < size; k++)
47       MPI_Send(&norm, 1, MPI_FLOAT, k, 0, MPI_COMM_WORLD);
48   } else {
49     MPI_Recv(&norm, 1, MPI_FLOAT, 0, 0, MPI_COMM_WORLD,
50              MPI_STATUS_IGNORE);
51   }
52
53   // step 4: every process normalizes its own block
54   for (int j = 0; j < n; j++) x[j] = x[j] / norm;
55   printf("process %d: norm = %.2f, x =", rank, norm);
56   for (int j = 0; j < n; j++) printf(" %.2f", x[j]);
57   printf("\n");
58
59   free(x);
60   MPI_Finalize();
61   return 0;
62 }
```

Outline

1. Processes, and the MPI library
2. Computing nine with three processes
3. Second example: normalizing a distributed vector
4. Summary
5. Compiling and running; the lab

The dictionary PRAM / OpenMP / CUDA / MPI

université PARIS-SACLAY

PRAM	**OpenMP** (session 3)	**CUDA** (session 5)	**MPI** (today)
forall $i = 1, \ldots, N$ **in parallel do** *body*	`#pragma omp parallel { body }`	kernel `f`, launched by `f<<<B, T>>>()`	`mpirun -np p ./prog`: the *body* is the whole program
N PUs	a few threads (one per core)	$B \times T$ threads, with the guard `if (i < n)`	p processes, one per core, on one machine or on many
index i of the PU	`omp_get_thread_num()`	`blockIdx.x * blockDim.x + threadIdx.x`	`rank` (`MPI_Comm_rank`)
number of PUs P	`omp_get_num_threads()`	`gridDim.x * blockDim.x`	`size` (`MPI_Comm_size`)
the shared memory	shared variables (declared outside)	GPU arrays, passed as pointers	**none**: a variable belongs to one process, the others see it only in a message
private registers	private variables (declared inside)	variables declared in the kernel	every variable
Bcast(x, N)	reading a shared scalar	a scalar kernel argument	$p - 1$ `MPI_Send` on the root, one `MPI_Recv` elsewhere
end of a step (lockstep)	`#pragma omp barrier`	the end of the kernel (one kernel per step)	an `MPI_Recv`: the receiver waits for the message; nothing else synchronizes
CRCW fusion write, or CREW + Reduce	two threads writing one variable: **a race**; own variable, then reduce	the same **race**: own cell, then reduce	no shared variable, **no race**: own value, then $p - 1$ `MPI_Recv` on the root

Outline

1. Processes, and the MPI library
2. Computing nine with three processes
3. Second example: normalizing a distributed vector
4. Summary
5. Compiling and running; the lab

Compiling and running an MPI program

- An MPI program is ordinary C++ with `#include <mpi.h>`, in a `.cpp` file. Compile it with `mpic++`, a wrapper that calls g++ with the paths and the library of MPI:

```
mpic++ -O2 prog.cpp -o prog
```

- Run it with `mpirun`, which launches the processes and connects them:

```
mpirun -np 3 ./prog    : three processes on this machine
```

  More processes than cores: `mpirun --oversubscribe -np 8 ./prog` (the option before `-np`); it works, no faster. `./prog` alone runs one process

- Installing Open MPI: Ubuntu `sudo apt install openmpi-bin libopenmpi-dev`, macOS `brew install open-mpi`, Windows through WSL: see the **installation guide**
- Compiler Explorer compiles and runs **one** program: it cannot launch `mpirun`. The lab runs on your own machine, Linux, Windows (WSL) or macOS, with Open MPI installed
- Today's examples: `make run-02-nine-rank` in `src/` (`NP=4` for four processes)

Lab 4: Distributed-memory programming with MPI

université PARIS-SACLAY

Environment

Your own machine, Linux, Windows (WSL) or macOS, with Open MPI installed (the installation guide): `mpic++ -O2 file.cpp -o prog`, then `mpirun -np 4 ./prog`. The six functions of today are the only ones you need. Write your answers as comments at the top of the files: the three `.cpp` files are what you submit.

Ex. 1: hello

`hello.cpp`: every process prints `I am process k of p`. Run it with `-np` 1, 2, 4, 8, twice each: what changes, what does not?

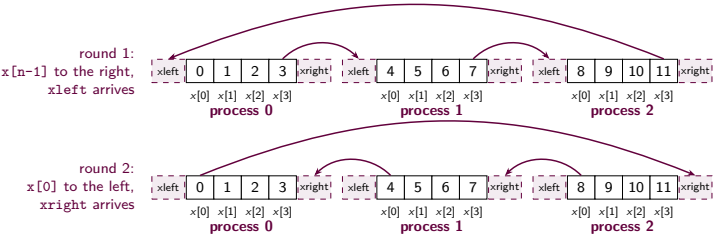
Ex. 2: 1-D convolution

`convolution.cpp`: a block `x` of n elements per process, a circular global vector of $N = nP$. $y[i] \leftarrow (gx[i-1] + gx[i] + gx[i+1])/3$: the two frontier elements come from the neighbours by messages, the rest is local

Ex. 3: mergesort

`mergesort.cpp`: a block of n integers per process, P a power of two. Sort it with the given `mergesort`; then fold the second half of the processes onto the first half, step by step, with sends, receives and the given `merge`: process 0 ends with the sorted array and checks it

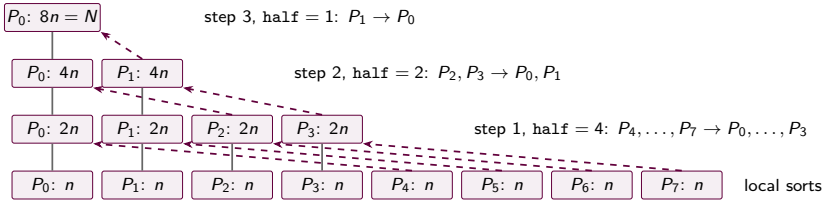
Exercise 2: 1-D convolution on a distributed vector



the cell i holds $gx[i]$; $N = 12$, $n = 4$, $P = 3$; the vector is circular: the left neighbour of $gx[0]$ is $gx[11]$

- **Data**: a block `x` of n elements per process, of a circular global vector gx of $N = nP$ elements ($gx[i] = i^2$ in the skeleton, which fills the blocks); nobody holds gx
- **Computation**: $y[j] = (x[j-1] + x[j] + x[j+1])/3$. The $n-2$ interior elements need only `x`; the first needs the last element of the left neighbour's block (`xleft`), the last the first element of the right neighbour's (`xright`)
- **Communication**: two messages per process, in two rounds: send `x[n-1]` to the right neighbour, receive `xleft` from the left one; then send `x[0]` left, receive `xright` from the right; neighbours modulo P

Exercise 3: parallel mergesort



- **Data**: $N = 2^{22}$ integers in P blocks, P a power of two and N a multiple of P: $n = N/P$ pseudo-random integers per process, in `a[0..n-1]`; `a` and `temp` have room for all N elements
- **Local sorts**: every process sorts its block with the given `mergesort`, no communication
- **Merging steps**: the second half of the active processes **folds onto the first half**. With 2 `half` active processes holding m sorted elements each: a process of rank $\geq$ `half` **sends** its m elements to process `rank` $-$ `half` and is done (dashed arrows); a process of rank $<$ `half` **receives** the m elements of process `rank` $+$ `half` right after its own and **merges** the two halves with the given `merge`: $2m$ elements, `half` processes left
- After $\log_2 P$ steps, process 0 holds the sorted array; the given `verify` checks it (sorted, same elements) and prints the speedup

Contact

Oguz Kaya
Université Paris-Saclay and LISN, Paris, France
oguz.kaya@universite-paris-saclay.fr
www.oguzkaya.com